

My formal teaching experience is as a teaching assistant (TA) at UIUC in numerical methods (400-level), numerical analysis (300-level), software design lab, and graduate software engineering. As a tutor in the CS Mentor Center at UT Dallas, I was responsible for every Freshman and Sophomore core CS class. In teaching outreach, I designed and instructed a course on Math of Music and Applied Geometry for high school students as part of the UIUC Principal Scholars Program and mentored a student in designing her own Interactive Fiction game in Girls Who Code. In more informal settings, I ran the GNU/Linux Users Group for a year. Outside of STEM, I taught Esperanto in an extracurricular group for a year.

The totality of my formal and informal teaching experience leads me to conclude that engaging students in critical thinking is the most important part of teaching.

The first component is that the students must be engaged.

I have lectured to groups as large as 75, and in my experience, I best kept students engaged by requiring some form of participation, especially verbal. When I was giving review sessions for the CS Mentor Center at UT Dallas, I passed a prop around the classroom, and whoever was holding the prop would have to answer the next question before passing it on. The prop gives everyone a chance to answer over the course of the year, not only the eager students who might otherwise dominate the conversation. Interactive electronic quizzes can also help maintain students' focus. Lastly, exuding enthusiasm as the lecturer can energize the students by proxy.

In smaller class settings, I have best engaged students by determining the students' interests, and making the class material relevant to these interests. When I taught the Math of Music in the Principal Scholars Program at UIUC, I asked the students for their favorite genres of music, favorite artists, and what instruments they play, so I could use examples from those genres, artists, and instruments. In a small group, I might even call out the student by name, "Joe could probably tell us that when he plays violin, he has to depress the neighboring strings to dampen the sympathetic vibrations." Mentioning their name triggers a neurological effect called the cocktail party effect, which snaps their attention back to the speaker. At cocktail parties, one needs to tune out all conversations other than one, and the human brain is quite adept at filtering those out; however, if one of those ignored conversations mentions the person's name, the sound of their name appears to often override the brain's default filtering, and brings the person's attention to that conversation immediately (Moray 1959).

In project mentoring, I have found that the most engaged students are those who research something they are passionate about. When I was a mentor for Girls Who Code, one student mastered the curriculum of the most advanced courses being offered. I sat the student down and talked to her about what she was interested in. She did not really need to learn more about C++ but was interested in making a text-based adventure game. I let her work on her game with minimal instruction. I viewed my role as to only intervene when she got stuck, and otherwise let her develop her game. I explained a concept if and when that concept became necessary to implement an aspect of her game.

The second component is critical thinking.

I view critical thinking as having two parts: deconstruction and construction. Deconstruction challenges assumptions entailed in the system, whereas in construction, one creates a new system by asserting new assumptions.

In a large class setting, one can encourage deconstruction by describing the limitations of what is taught. When I learned Dijkstra's Algorithm, it felt like the shortest path problem was totally 'solved' and therefore boring. When I teach Dijkstra's Algorithm, I might discuss theoretical

limitations (e.g., negative edges) or even practical limitations that make it infeasible for use in Google Maps. An overlooked part of 'learning' something is to understand when to not use that thing.

Programming is more properly thought of as theory construction (Naur 1985), so programming projects can be a good opportunity for construction in large class settings. What separates the good programming assignments is whether they empower students to make design decisions. Besides the opportunity to make their own designs, programming assignments should offer students the opportunity to practice pitching their designs to non-technical audiences, an invaluable skill in industry.

In smaller class settings, Socratic dialogues can encourage critical thinking. The teacher will pose questions, and the students will try to construct answers. The teacher probes the given answers (deconstruction), and the group (students and teacher) can collectively refine the answers, leading to a more complete synthesis of knowledge. I conduct such activities in the Linux Users Group, where we can debate such subjects as whether microkernels are superior (however that is defined) to modular kernels. The goal is not to have everyone come to the same opinion, but for each to learn more about the others' beliefs.

In project mentoring, building a project is inherently constructive. However, practicing destruction can make one better at construction. To that end, I love to guide students through deconstructing a research paper. Even IEEE Test of Time award winning papers can be picked apart with enough creativity. Students should feel that assumptions are there to be challenged, which leads to new ways of construction. At the same time, when everything can be deconstructed, it takes experience and thoughtfulness to decide what will be most useful to deconstruct. Students should learn that their own research projects are not worthless if they have limitations; everything has limitations. The best thing to do is to simply acknowledge and qualify the limitations.

AI

changes the effective means of instruction and objectives of instruction. Evaluating the work of others is a more important objective, when LLMs can generate new work instantly. The skill of destruction transfers over, as the operator would deconstruct assumptions the LLM may have made, searching for what is ungrounded.

For lower-level classes, I prefer to limit the use of AI. There is value in learning how to do something by hand once, so one knows how it works in principle, even if that thing can be automated easily. We learn long-division by hand with integers, even though integer division can be automated, so that we can intuit the algorithm and the meaning of polynomial long-division. AI may be limited by in-person tests or on-camera remote tests.

For upper-level classes, the point is not the code, but rather the design. Students should practice using LLMs, which are adept at generating code, but need to be told what code to generate. Students are ultimately responsible for their code, however, and should be able to clearly explain the code, clearly state its limitations, and accurately predict how it will behave in edge-cases, whether they used AI or not. As the 1979 IBM manual stated, "A computer can never be held accountable. Therefore, a computer must never make a management decision."

What I could teach

My research is in software engineering, so I could teach those at a graduate level, and I already have TA experience in graduate-level software engineering. My research utilizes concepts from operating systems, discrete math, algorithms, and data science, so I could teach those at a Junior and Senior level. In my undergraduate institution, I worked in the CS Mentor Center, where I was responsible

for tutoring walk-ins for any Freshman and Sophomore level core CS course, so I feel I could teach any Freshman and Sophomore level core CS class.

If I had the freedom to teach whatever I wanted, I would teach a course on “applied algorithms.” I would explain a practical tool and then explain the theory behind it. Students gain knowledge of practical tools that are often described as a “missing semester” while also seeing how OS theory, statistics, and complexity theory applies in the real world.

Practical tool	Underlying theory
UNIX pipes	Queuing theory, dining philosophers, c10k problem
Git	Algebraic lattices
Apt-get	SAT solving, NP-completeness
Redis	Knapsack problem, NP-completeness
Async/await in JS, Python, or Rust	Monads, green threading
QEMU	Universal Turing machine, Turing completeness
Tar/gzip	Kolmogorov complexity
SSHFS	FUSE/Microkernels